Datenbanken: Datenintegrität

Definition "Datenkonsistenz"

"in der Datenbankorganisation (...) die **Korrektheit der gespeicherten Daten** im Sinn einer **widerspruchsfreien und vollständigen Abbildung** der relevanten Aspekte des erfassten Realitätsausschnitts."

http://wirtschaftslexikon.gabler.de/Archiv/57132/datenintegritaet-v6.html
Hervorhebung hinzugefügt

→ *logisch korrekter Zustand der Daten*

Inkonsistente / konsistente Daten

kunden

idKunde	name	postleitzahl	ortName	arbeitgeber
1	Schmitt	10000	Musterhausen	Bäckerei Zimmermann GmbH
2	Müller	10000	Musterausen	Schlüter & Co KG
	Müller	79132	Coburg	Bäckerei Zimmermann

Welche Fehler erkennen Sie?

(abgesehen von der Verletzung einer Normalform)

Inkonsistente / konsistente Daten

kunden

idKunde	name	postleitzahl	ortName	arbeitgeber
1	Schmitt	10000	Musterhausen	Bäckerei Zimmermann GmbH
2	Müller	10000	Musterausen	Schlüter & Co KG
	Müller	79132	Coburg	Bäckerei Zimmermann

Inkonsistente / konsistente Daten

kunden

idKunde	name	postleitzahl	arbeitgeberFK
1	Schmitt	10000	1
2	Müller	10000	14
3	Maier	79312	7

(FK = Foreign Key, Fremdschlüssel)

orte

postleitzahl	name
10000	Musterhausen
79098	Freiburg

Welche Fehler erkennen Sie?

Inkonsistente / konsistente Daten

kunden

idKunde	name	postleitzahl	arbeitgeberFK
1	Schmitt	10000	1
2	Müller	10000	14
3	Maier	79312	7

orte

postleitzahl	name
10000	Musterhausen
79098	Freiburg

arbeitgeber

idArbeitgeber	name
1	Bäckerei Zimmermann
7	Hug GmbH
14	ForSi

Integritätsregeln

TBZ

Datenkonsistenz/Datenintegrität wird gewährleistet durch Integritätsbedingungen:

Domänenintegrität/Bereichsintegrität:
Attribute sind nur gültig, wenn sie einen bestimmten Wertebereich haben

id	name	geburtsdatum
1	Smith	hihihi

Entitätsintegrität
Jeder Datensatz ist eindeutig definiert (z.B. durch PRIMARY KEY).

id	name	geburtsdatum
1	Smith	2012-12-12
1	Sponz	2012-12-13

Referentielle Integrität
Beziehungen zwischen Tabellen müssen synchronisiert bleiben. (s.u.)

Benutzerdefinierte Integrität
Sonstige vom Benutzer festgelegte Regeln
(z.B.: Datum darf nicht vor 01.01.2000 liegen)

Integritätsregeln

Datenkonsistenz/Datenintegrität wird gewährleistet durch Integritätsbedingungen:

Domänenintegrität/Bereichsintegrität:

Entitätsintegrität

Referentielle Integrität

Benutzerdefinierte Integrität

Sind diese Bedingungen erfüllt, ist die Datenbank konsistent.

Referentielle Integrität

TBZ

Primärschlüssel-Fremdschlüssel-Beziehungen müssen intakt sein:

kunden

idKunde	name	postleitzahl	arbeitgeberFK
1	Schmitt	10000	1
2	Müller	10000	14
3	Maier	79312	7

Keine passende Tabelle
→ kein PK zu den FKs

Kein passender Datensatz
→ kein PK zum FK

orte

postleitzahl	name
10000	Musterhausen
79098	Freiburg

möglicherweise nett fürs Abi ☺

Definition
"Die referentielle Integrität (auch Beziehungsintegrität) besagt, dass Attributwerte eines Fremdschlüssels auch als Attributwert des Primärschlüssels vorhanden sein müssen."

http://de.wikipedia.org/wiki/Referentielle_Integrität

Referentielle Integrität

TBZ

Primärschlüssel-Fremdschlüssel-Beziehungen müssen intakt sein:

kunden

idKunde	name	postleitzahl	arbeitgeberFK
1	Schmitt	10000	1
2	Müller	10000	14
3	Maier	79312	7

Keine passende Tabelle
→ kein PK zu den FKs

Kein passender Datensatz
→ kein PK zum FK

orte

postleitzahl	name
10000	Musterhausen
79098	Freiburg

Grundregel

Fremdschlüssel müssen IMMER auf existierende Datensätze verweisen!

Referentielle Integrität

TBZ

Grundregel

Fremdschlüssel müssen IMMER auf existierende Datensätze verweisen!

kunden

idKunde	name	postleitzahl
1	Schmitt	10000
2	Müller	10000
3	Maier	79312

muss beachtet werden bei ...

1. Löschen von Datensätzen oder Tabellen

orte

postleitzahl	name
~~10000~~	~~Musterhausen~~
79312	Emmendingen

Löschen dieses Datensatzes führt zu Inkonsistenzen (da zum FK "kunden.postleitzahl" kein Datensatz existiert)

Referentielle Integrität

TBZ

Grundregel

Fremdschlüssel müssen IMMER auf existierende Datensätze verweisen!

kunden

idKunde	name	postleitzahl
1	Schmitt	10000
2	Müller	10000
3	Maier	79312
4	Huber	80985

muss beachtet werden bei ...

2. Einfügen von Datensätzen

orte

postleitzahl	name
10000	Musterhausen
79312	Emmendingen

Einfügen dieses Datensatzes führt zu Inkonsistenzen (da zum FK "kunden.postleitzahl" kein Datensatz existiert)

Aufgaben

artikel

artikelID	name
1	Hut
2	Schirm

Matching-Tabelle
(n:m)

artikel_hat_lieferant

artikelID	lieferantID
1	1
1	2
2	1

lieferanten

lieferantID	name
1	Smith GmbH
2	John D.

Aufgaben

artikel

artikelID	name
1	Hut
2	Schirm
3	Schuh

(a)

artikel_hat_lieferant

artikelID	lieferantID
1	1
~~1~~	~~2~~
2	1
2	3

(c)

(b)

lieferanten

lieferantID	name
1	Smith GmbH
~~2~~	~~John D.~~

(d)

Welche Operationen sind zulässig?

a) Einfügen Datensatz
b) Einfügen Datensatz
c) Löschen Datensatz
d) Löschen Datensatz

LÖSUNG

artikel

artikelID	name
1	Hut
2	Schirm
3	Schuh

(a)

artikel_hat_lieferant

artikelID	lieferantID
1	1
~~1~~	~~2~~
2	1
2	3

(c)

(b)

lieferanten

lieferantID	name
1	Smith GmbH
~~2~~	~~John D.~~

(d)

Welche Operationen sind zulässig?

a) Einfügen Datensatz - JA
b) Einfügen Datensatz – JA (wenn zugehörige Datensätze vorhanden)
c) Löschen Datensatz - JA
d) Löschen Datensatz – NEIN (sofern Datensatz in Kindtabelle vorhanden)

Änderungsweitergabe, Löschweitergabe; MySQL: FOREIGN-KEY-CONSTRAINTS

TBZ

DBMS kann helfen, Datenintegrität zu wahren, z.B. über Einschränkungen (CONSTRAINTS):

- Änderungsweitergabe
- Löschweitergabe

Achtung:
InnoDB erforderlich!

Löschweitergabe

Löschen wir einen Ort, dann werden alle Kunden in diesem Ort gelöscht.

kunden

id	name	postleitzahl_FK
1	Schmitt	10000
2	Müller	10000
3	Maier	79312

orte

postleitzahl	name
10000	Musterhausen
79312	Freiburg

löschen

kunden

id	name	postleitzahl_FK
3	Maier	9312

automatische Löschung in Kindtabelle

orte

postleitzahl	name
79312	Freiburg

(Das ist natürlich nicht sinnvoll; besseres Beispiel: Wird ein Lieferant gelöscht, dann könnte man auch die von ihm gelieferten Artikel löschen.

Änderungsweitergabe

Ändert sich die PLZ von Musterhausen (Elterntabelle "orte"), wird diese Änderung an die Kindtabelle weitergegeben.

kunden

id	name	postleitzahl_FK
1	Schmitt	10000
2	Müller	10000
3	Maier	79312

orte

postleitzahl	name
10000	Musterhausen
79312	Freiburg

Änderung

kunden

id	name	postleitzahl_FK
1	Schmitt	99999
2	Müller	99999
3	Maier	79312

automatische Änderung in Kindtabelle

orte

postleitzahl	name
99999	Musterhausen
79312	Freiburg

Änderungsweitergabe

Ändert sich die PLZ von Musterhausen (Elterntabelle "orte"), wird diese Änderung an die Kindtabelle weitergegeben.

kunden

id	name	postleitzahl_FK
1	Schmitt	10000
2	Müller	10000
3	Maier	79312

kunden

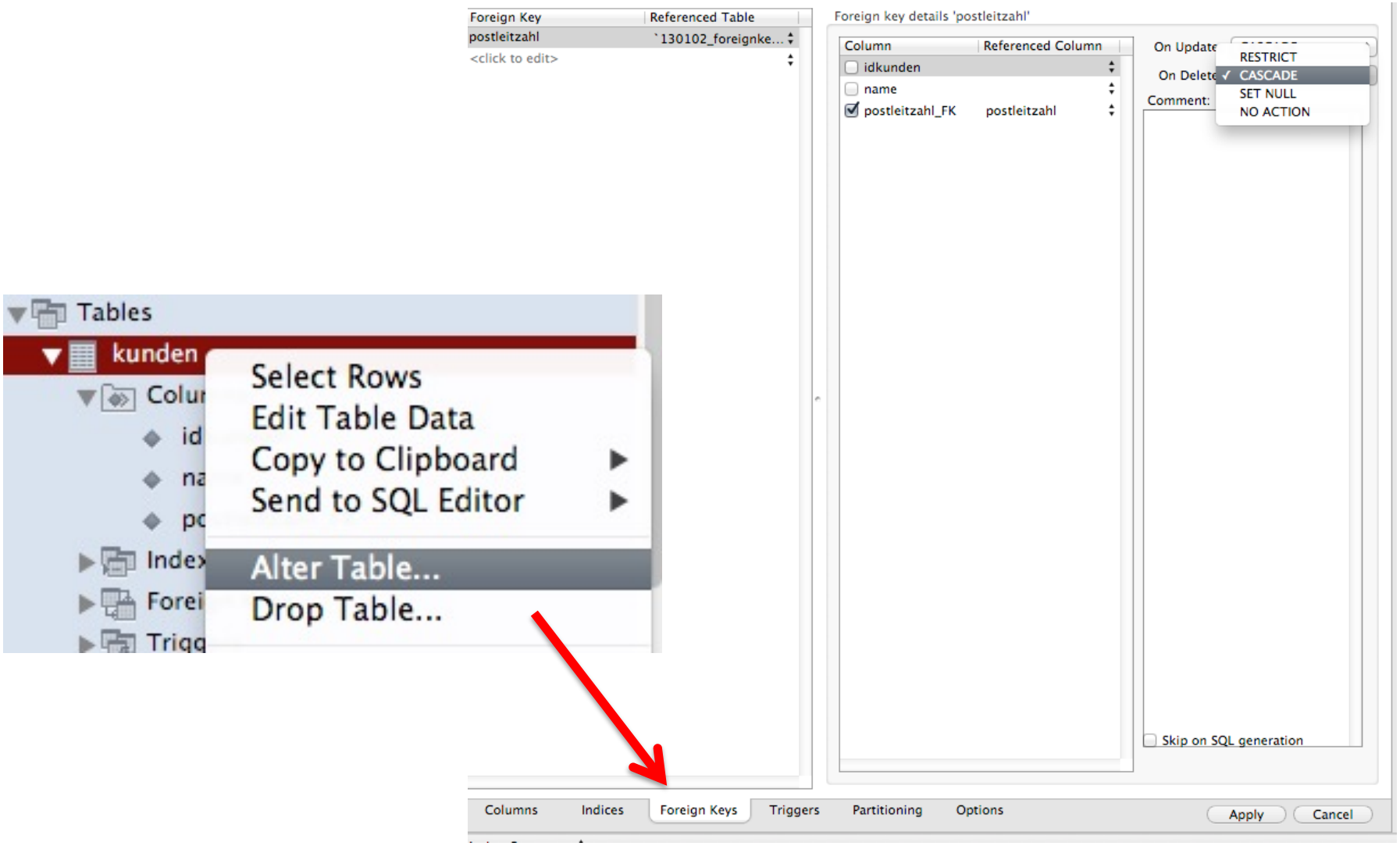
id	name	postleitzahl_FK
1	Schmitt	99999
2	Müller	99999
3	Maier	79312

automatische Änderung in Kindtabelle

```
CREATE  TABLE kunden (
  id INT, name VARCHAR(45),postleitzahl_FK VARCHAR(5),
  PRIMARY KEY (idkunden), -- ....
    CONSTRAINT FK_postleitzahl
        FOREIGN KEY (postleitzahl_FK)
              REFERENCES orte (postleitzahl)
        ON DELETE CASCADE
        ON UPDATE CASCADE)
ENGINE = InnoDB;
```

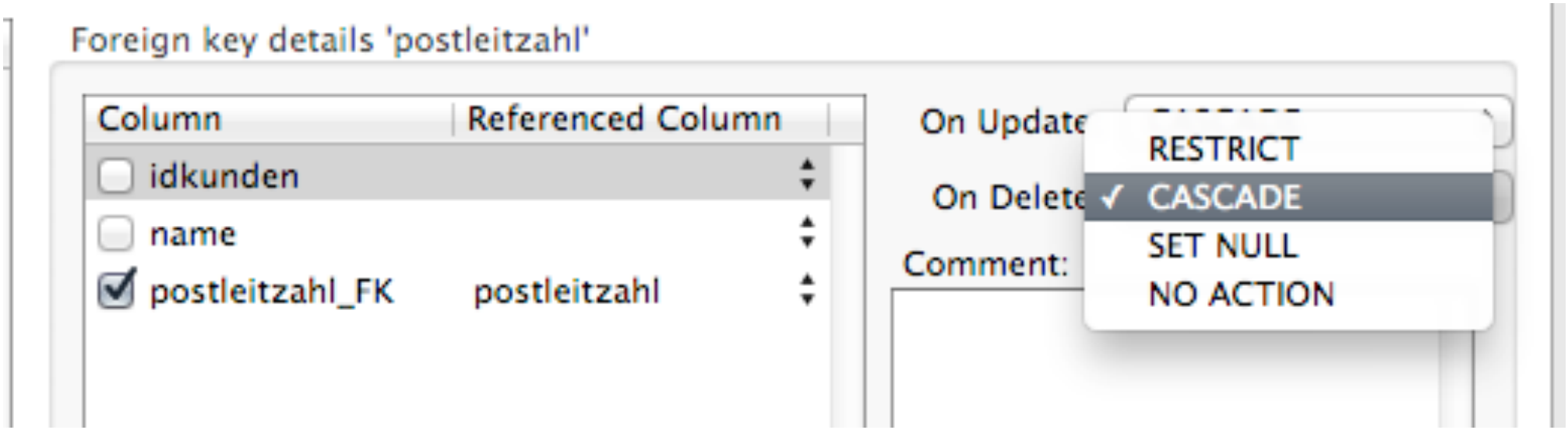
Änderungsweitergabe

Ändert sich die PLZ von Musterhausen (Elterntabelle "orte"), wird diese Änderung an die Kindtabelle weitergegeben.



Änderungsweitergabe

Ändert sich die PLZ von Musterhausen (Elterntabelle "orte"), wird diese Änderung an die Kindtabelle weitergegeben.



FOREIGN KEY-CONSTRAINTS

MySQL/InnoDB

```
ON UPDATE ...
ON DELETE ...
```

CASCADE	Änderungen/Löschung auch in Kindtabelle vornehmen
SET NULL	Fremdschlüssel in Kindtabelle auf NULL setzen
NO ACTION	keine weitere Aktion
RESTRICT	Aktion verweigern

⇒Gleiche Verhalten

FOREIGN KEY-CONSTRAINTS

MySQL/InnoDB

```
ON UPDATE ...
ON DELETE ...
```

CASCADE	Änderungen/Löschung auch in Kindtabelle vornehmen
SET NULL	Fremdschlüssel in Kindtabelle auf NULL setzen
NO ACTION	keine weitere Aktion
RESTRICT	

```
CREATE  TABLE kunden (
  id INT, name VARCHAR(45),postleitzahl_FK VARCHAR(5),
  PRIMARY KEY (idkunden), -- ....
    CONSTRAINT FK_postleitzahl
        FOREIGN KEY (postleitzahl_FK)
            REFERENCES orte (postleitzahl)
        ON DELETE RESTRICT
        ON UPDATE No ACTION)
ENGINE = InnoDB;
```